



7.2. Data Concatenation, Comparison, and Segmentation

Previous Section:

 [7.1. Data Merging & Joining](#)

Contents:

[1. Concatenating DataFrames](#)

[Example 1: Vertical Concatenation](#)

[Example 2: Horizontal Concatenation](#)

[2. Comparing Two DataFrames](#)

[3. Data Segmentation](#)

[Example 1: Segmenting Age](#)

[Example 2: Segmenting GPA](#)

[Summary](#)

[References](#)

In addition to merging and joining datasets, Pandas provides several other useful operations for combining, comparing, and reorganizing data. We may want to stack multiple datasets together, place them side-by-side, identify differences between two versions of the same dataset, or group continuous numerical values into meaningful categories.

In this section, we create two datasets containing students from different majors:

- **studentAI** – Students majoring in Artificial Intelligence.

- **studentCSC** – Students majoring in Computer Science.

Each dataset contains **20 samples** with the columns **Name**, **Age**, and **GPA**.

```
import numpy as np
import pandas as pd

np.random.seed(42)

studentAI = pd.DataFrame({
    "Name": [f"AI_{i}" for i in range(1,21)],
    "Age": np.random.randint(18,22,20),
    "GPA": np.round(np.random.uniform(2.0,4.0,20),2)
})

studentCSC = pd.DataFrame({
    "Name": [f"CSC_{i}" for i in range(1,21)],
    "Age": np.random.randint(18,22,20),
    "GPA": np.round(np.random.uniform(2.0,4.0,20),2)
})

print(studentAI.head())
print(studentCSC.head())
```

1. Concatenating DataFrames

Pandas provides the `pd.concat()` function to combine multiple DataFrames.

There are two common ways to concatenate data:

- **Row-wise (Vertical):** Stack one dataset below another. This is commonly used when combining datasets with identical columns.
- **Column-wise (Horizontal):** Place datasets side-by-side. Unlike `merge()`, the DataFrames do not need a common key, but they should have the same number of rows or compatible indexes.

Example 1: Vertical Concatenation

```
result = pd.concat(  
    [studentAI, studentCSC],  
    axis=0,  
    ignore_index=True  
)  
  
print(result.head())
```

Example 2: Horizontal Concatenation

```
result = pd.concat(  
    [studentAI, studentCSC],  
    axis=1  
)  
  
print(result.head())
```

Note: Horizontal concatenation simply places columns next to one another based on their indexes. It does **not** attempt to find matching rows like `merge()`.

2. Comparing Two DataFrames

During data cleaning or preprocessing, it is common to compare two similar DataFrames to determine which values have changed. Pandas provides the `compare()` method to highlight these differences.

```
studentAI2 = studentAI.copy()  
  
studentAI2.loc[0, "GPA"] = 3.95  
studentAI2.loc[3, "Age"] = 21  
studentAI2.loc[5, "GPA"] = 2.50  
  
comparison = studentAI.compare(studentAI2)
```

```
print(comparison)
```

The output displays only the rows and columns where differences exist, making it useful for validating preprocessing steps or detecting modifications.

3. Data Segmentation

Sometimes it is useful to convert continuous numerical values into categorical groups. This process is known as **segmentation** or **binning**.

Pandas provides the `pd.cut()` function to divide numerical values into intervals.

Example 1: Segmenting Age

```
studentAI["Age Group"] = pd.cut(
    studentAI["Age"],
    bins=[17, 18, 20, 22],
    labels=["18", "19-20", "21"]
)

print(studentAI.head())
```

Example 2: Segmenting GPA

```
studentAI["Performance"] = pd.cut(
    studentAI["GPA"],
    bins=[0, 2.5, 3.2, 4.0],
    labels=["Low", "Average", "High"]
)

print(studentAI.head())
```

Segmenting numerical features can make datasets easier to interpret and is particularly useful for **categorical feature-based models**, such as **Decision**

Trees and **Random Forests**, where grouped features may improve interpretability.

Summary

Pandas provides several flexible tools for combining and organizing datasets.

- `pd.concat()` combines multiple DataFrames by stacking rows or columns.
- `compare()` identifies differences between two similar DataFrames.
- `pd.cut()` segments continuous numerical values into categorical intervals.

Together with `merge()` and `join()`, these functions provide a comprehensive toolkit for integrating, comparing, and restructuring datasets before performing analysis or building machine learning models.

References

https://pandas.pydata.org/docs/user_guide/merging.html

<https://www.geeksforgeeks.org/pandas/python-pandas-merging-joining-and-concatenating/>

<https://www.geeksforgeeks.org/pandas/how-to-compare-two-dataframes-with-pandas-compare/>

Next Section:

 [8. Data Visualization](#)